



Vietnam National University HCMC
International University

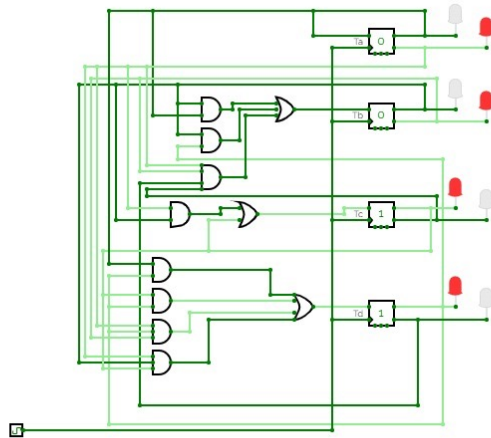


School of
Electrical Engineering

EE053IU

Digital Logic Design

Lecture 2: Number Systems, Operations, and Codes



INSTRUCTOR: Dr. Vuong Quoc Bao

Lecture Objectives

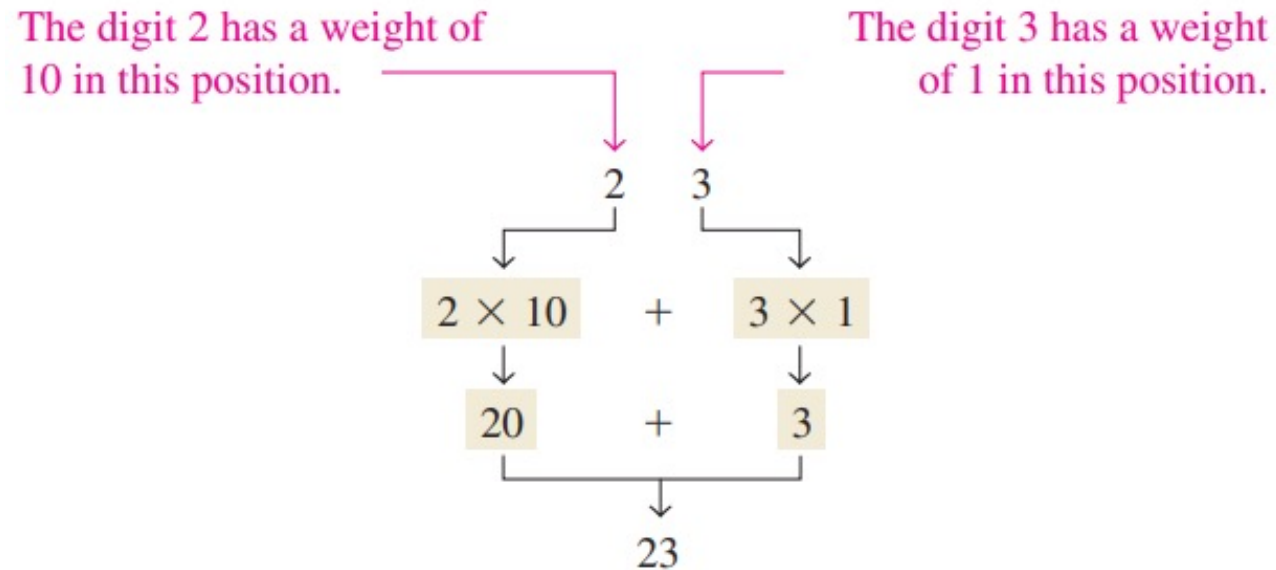
- Review the decimal number system
- Count in the binary number system
- Convert from decimal to binary and from binary to decimal
- Apply arithmetic operations to binary number
- Determine the 1's and 2's complements of a binary number
- Express signed binary numbers in sign-magnitude, 1's complement, 2's complement, and floating-point format
- Carry out arithmetic operations with signed binary numbers

Lecture Objectives

- Convert between the binary and hexadecimal number systems
- Add numbers in hexadecimal form
- Convert between the binary and octal number systems
- Express decimal numbers in binary coded decimal (BCD) form
- Add BCD numbers
- Convert between the binary system and the Gray code
- Interpret the American Standard Code for Information Interchange (ASCII)

I. Decimal Numbers

- In the decimal number system each of the ten digits, 0 through 9, represents a certain quantity.



- The decimal number system has a base of 10.

$$\dots 10^5 10^4 10^3 10^2 10^1 10^0$$

- The value of a digit is determined by its position in the number.

$$10^2 10^1 10^0 . 10^{-1} 10^{-2} 10^{-3} \dots$$

↑ Decimal point

EXAMPLE 2-1

Express the decimal number **47** as a sum of the values of each digit.

1. Digits and Their Positions

4 **7**



Tens place
(10^1)
Weight = 10



Ones place
(10^0)
Weight = 1

2. Expanded Form

$$47 = (4 \times 10^1) + (7 \times 10^0)$$



$$\begin{aligned} 4 \times 10^1 \\ &= 4 \times 10 \\ &= 40 \end{aligned}$$



$$\begin{aligned} 7 \times 10^0 \\ &= 7 \times 1 \\ &= 7 \end{aligned}$$

3. Result

$$\begin{aligned} 47 &= (4 \times 10^1) + (7 \times 10^0) \\ &= (4 \times 10) + (7 \times 1) \end{aligned}$$

$$= 40 + 7$$



Related Problem

Determine the value of each digit in **939**.

What is the value of 9, 3, and 9?



EXAMPLE 2-2

Express the decimal number 568.23 as a sum of the values of each digit.

1. Digits and Their Positions

5	6	8	.	2	3
↓	↓	↓		↓	↓
Hundreds	Tens	Ones		Tenths	Hundredths
10^2	10^1	10^0		10^{-1}	10^{-2}
100	10	1		0.1	0.01

2. Expanded Form

$$\begin{aligned} 568.23 &= (5 \times 10^2) + (6 \times 10^1) + (8 \times 10^0) + (2 \times 10^{-1}) + (3 \times 10^{-2}) \\ &= (5 \times 100) + (6 \times 10) + (8 \times 1) + (2 \times 0.1) + (3 \times 0.01) \end{aligned}$$

$$\begin{aligned} 5 \times 100 \\ = 500 \end{aligned}$$

$$\begin{aligned} 6 \times 10 \\ = 60 \end{aligned}$$

$$\begin{aligned} 8 \times 1 \\ = 8 \end{aligned}$$

$$\begin{aligned} 2 \times 0.1 \\ = 0.2 \end{aligned}$$

$$\begin{aligned} 3 \times 0.01 \\ = 0.03 \end{aligned}$$

$$568.23 = 500 + 60 + 8 + 0.2 + 0.03$$



Related Problem

Determine the value of each digit in 67.924.

II. Binary Numbers

- The binary number system has two digits (bits).
- The decimal number system has a base of 2.

Counting in Binary

- In general, with n bits you can count up to a number equal to $2^n - 1$.
- Largest decimal number = $2^n - 1$

TABLE 2-1

Decimal Number	Binary Number			
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1
10	1	0	1	0
11	1	0	1	1
12	1	1	0	0
13	1	1	0	1
14	1	1	1	0
15	1	1	1	1

An Application

Learning to count in binary will help you to basically understand how digital circuits can be used to count events. Let's take a simple example of counting tennis balls going into a box from a conveyor belt. Assume that nine balls are to go into each box.

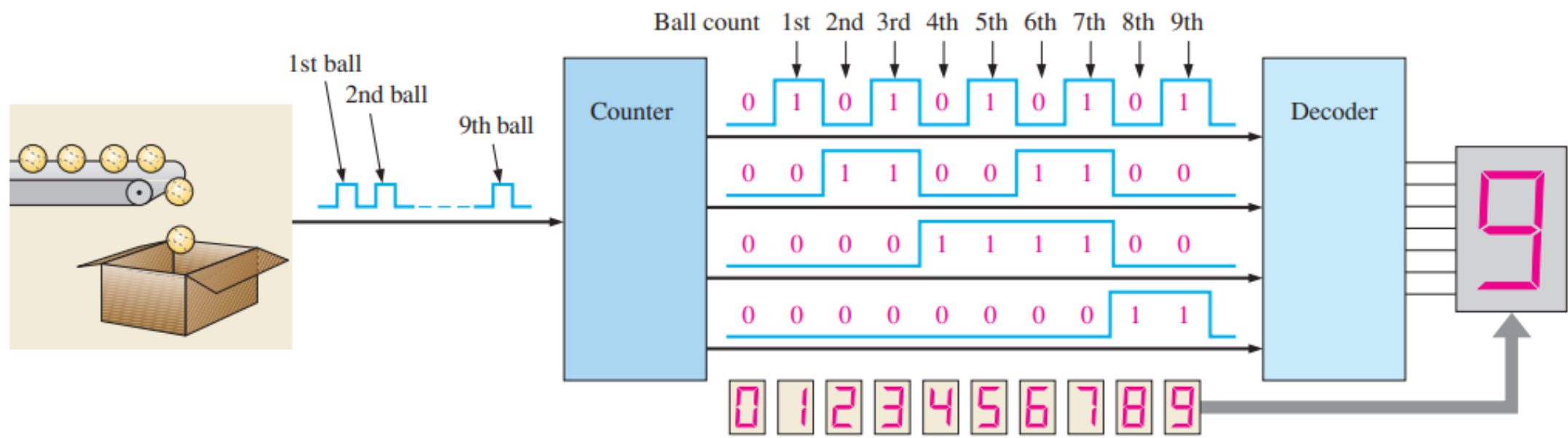


FIGURE 2-1 Illustration of a simple binary counting application.

The Weighting Structure of Binary Numbers

- A binary number is a weighted number. The right-most bit is the **LSB (least significant bit)** in a binary whole number and has a weight of $2^0 = 1$.
- The weights increase from right to left by a power of two for each bit. The left-most bit is the **MSB (most significant bit)**; its weight depends on the size of the binary number.
- *The weight structure of a binary number is*

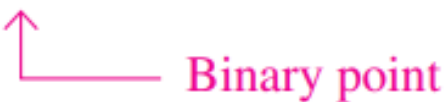
$$2^{n-1} \dots 2^3 2^2 2^1 2^0 . 2^{-1} 2^{-2} \dots 2^{-n}$$


TABLE 2-2														
Binary weights.														
Positive Powers of Two (Whole Numbers)									Negative Powers of Two (Fractional Number)					
2 ⁸	2 ⁷	2 ⁶	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰	2 ⁻¹	2 ⁻²	2 ⁻³	2 ⁻⁴	2 ⁻⁵	2 ⁻⁶
256	128	64	32	16	8	4	2	1	1/2	1/4	1/8	1/16	1/32	1/64
									0.5	0.25	0.125	0.625	0.03125	0.015625

Binary-to-Decimal Conversion

- The decimal value of any binary number can be found by adding the weights of all bits that are 1 and discarding the weights of all bits that are 0.

EXAMPLE 2-3

Convert the binary whole number 1101101 to decimal.

1 Write the bit positions and weights

Bit position	6	5	4	3	2	1	0
Weight (2^n)	2^6	2^5	2^4	2^3	2^2	2^1	2^0
	64	32	16	8	4	2	1
Binary number	1	1	0	1	1	0	1

2 Sum the weights for bits that are 1

$$\begin{aligned} 1101101 &= 2^6 + 2^5 + 2^3 + 2^2 + 2^0 \\ &= 64 + 32 + 8 + 4 + 1 \\ &= \mathbf{109} \end{aligned}$$



Related Problem

Convert the binary number 10010001 to decimal.

Binary-to-Decimal Conversion

- The decimal value of any binary number can be found by adding the weights of all bits that are 1 and discarding the weights of all bits that are 0.

EXAMPLE 2-4

Convert the fractional binary number 0.1011 to decimal.

1

Bit positions and weights

Bit position	-1	-2	-3	-4	
Weight (2 ⁿ)	2 ⁻¹	2 ⁻²	2 ⁻³	2 ⁻⁴	
Decimal value	0.5	0.25	0.125	0.0625	
Binary number	0	1	0	1	1

2

Sum the weights for bits that are 1

$$\begin{aligned} 0.1011 &= 2^{-1} + 2^{-3} + 2^{-4} \\ &= 0.5 + 0.125 + 0.0625 \\ &= \mathbf{0.6875} \end{aligned}$$

Result

$$0.1011_{(2)} = \mathbf{0.6875}_{(10)}$$



Related Problem

Convert the binary number 10.111 to decimal.

3. Decimal-to-Binary Conversion

Sum-of-Weights Method

- To get the binary number for a given decimal number, find the binary weights that add up to the decimal number.
- The decimal number 9, for example, can be expressed as the sum of binary weights as follows:

$$9 = 8 + 1 \quad \text{or} \quad 9 = 2^3 + 2^0$$

- Placing 1s in the appropriate weight positions, 2^3 and 2^0 , and 0s in the 2^2 and 2^1 positions determines the binary number for decimal 9.

2^3	2^2	2^1	2^0	
1	0	0	1	Binary number for decimal 9

Sum-of-Weights Method

EXAMPLE 2–5 Convert the following decimal numbers to binary.

Solutions

- (a) $12 = 8 + 4 = 2^3 + 2^2 \longrightarrow 1100$
- (b) $25 = 16 + 8 + 1 = 2^4 + 2^3 + 2^0 \longrightarrow 11001$
- (c) $58 = 32 + 16 + 8 + 2 = 2^5 + 2^4 + 2^3 + 2^1 \longrightarrow 111010$
- (d) $82 = 64 + 16 + 2 = 2^6 + 2^4 + 2^1 \longrightarrow 1010010$

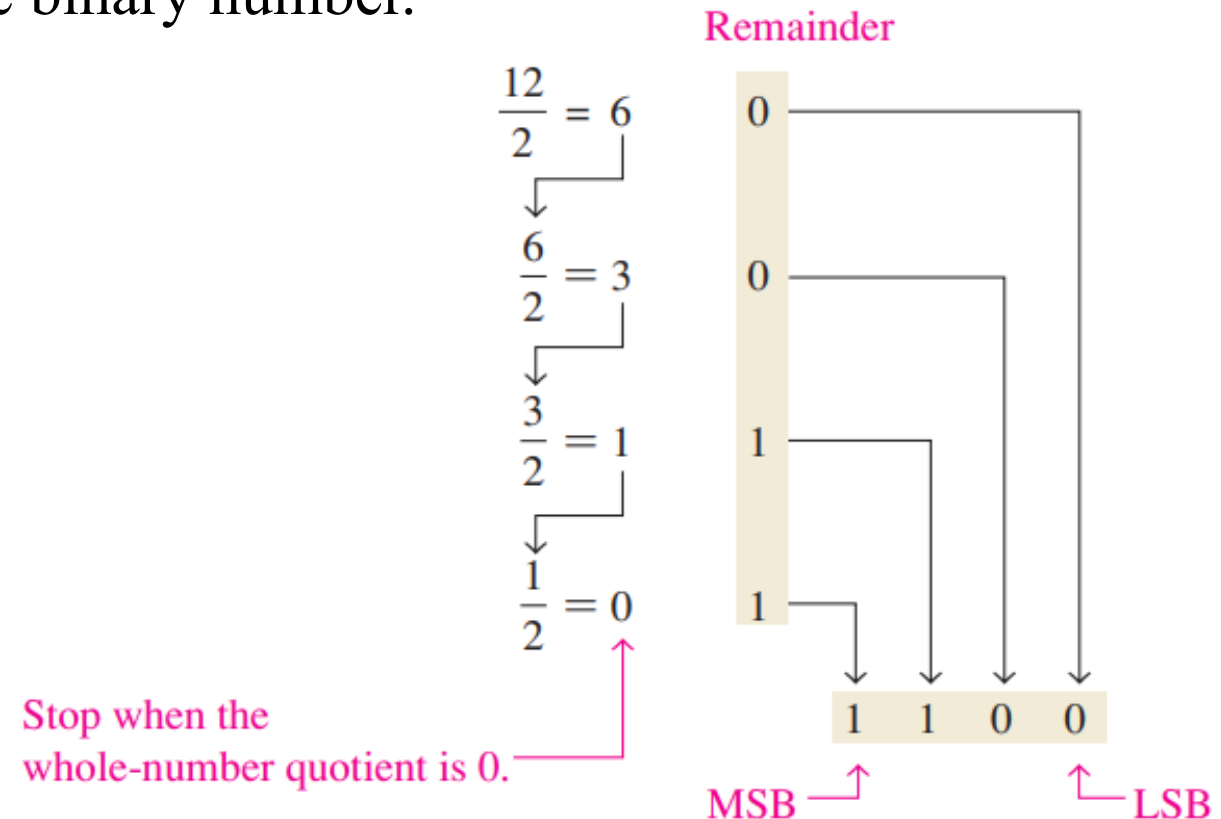


Related Problem

Convert the decimal number 125 to binary.

Repeated Division-by-2 Method

- A systematic method of converting whole numbers from decimal to binary is the repeated division-by-2 process.
- Then divide each resulting quotient by 2 until there is a 0 whole-number quotient. The remainders generated by each division form the binary number.
- The first remainder to be produced is the **LSB (least significant bit)** in the binary number, and the last remainder to be produced is the **MSB (most significant bit)**.



EXAMPLE 2–6

Convert the following decimal numbers to binary: (a) 19 (b) 45

(a) Convert 19 to binary.

Division by 2	Quotient	Remainder
$19 \div 2$	9	1
$9 \div 2$	4	1
$4 \div 2$	2	0
$2 \div 2$	1	0
$1 \div 2$	0	1

Remainder
(from bottom to top)

1
1
0
0
1



$$19_{10} = 10011_2$$

↑
MSB↑
LSB**(b)** Convert 45 to binary.

Division by 2	Quotient	Remainder
$45 \div 2$	22	1
$22 \div 2$	11	0
$11 \div 2$	5	1
$5 \div 2$	2	1
$2 \div 2$	1	0
$1 \div 2$	0	1

Remainder
(from bottom to top)

1
0
1
1
0
1



$$45_{10} = 101101_2$$

↑
MSB↑
LSB**Related Problem**

Convert decimal number 39 to binary.

Converting Decimal Fractions to Binary

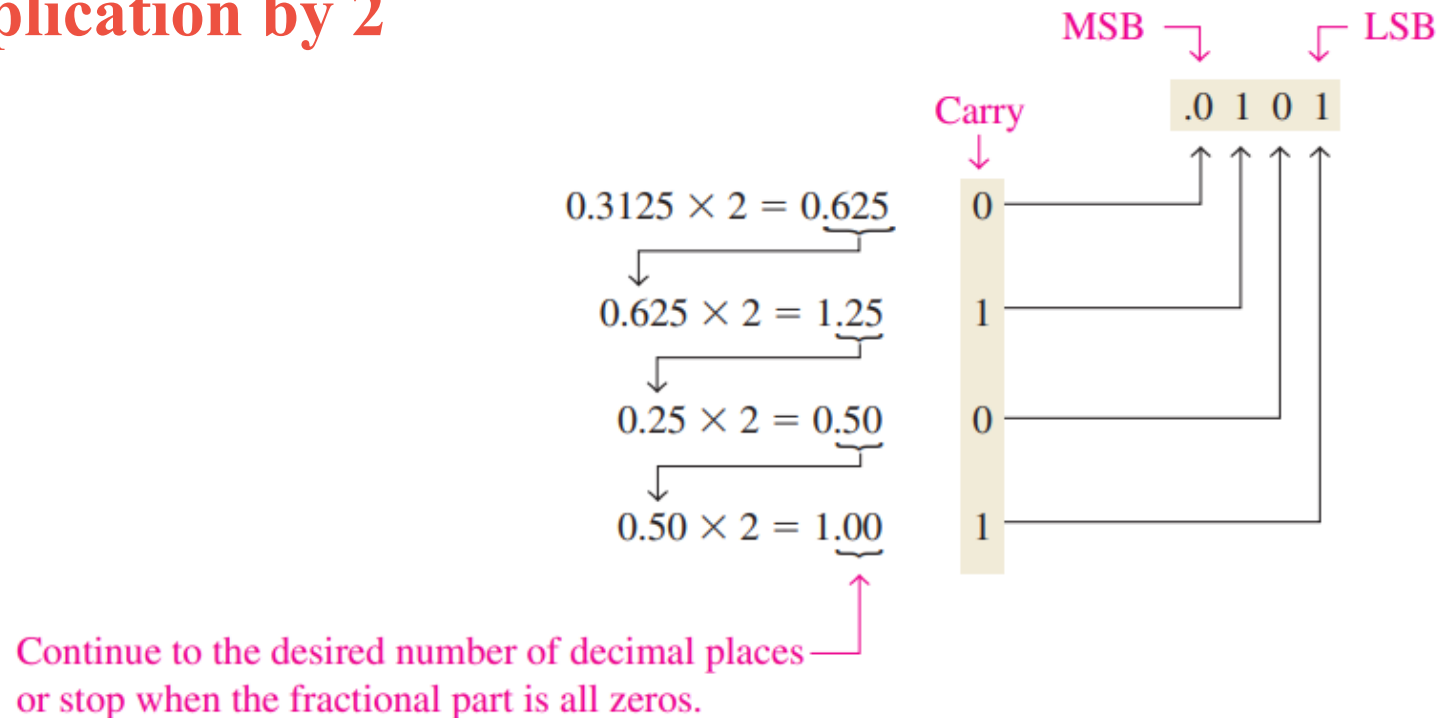
Sum-of-Weights

- The sum-of-weights method can be applied to fractional decimal numbers, as shown in the following example:

$$0.625 = 0.5 + 0.125 = 2^{-1} + 2^{-3} = 0.101$$

- There is a 1 in the 2^{-1} position, a 0 in the 2^{-2} position, and a 1 in the 2^{-3} position.

Repeated Multiplication by 2

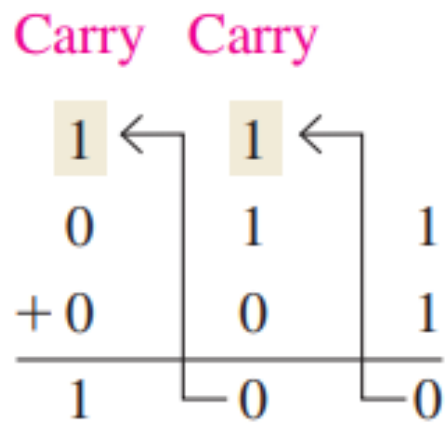


4. Binary Arithmetic

Binary Addition

The four basic rules for adding binary digits (bits) are as follows:

$0 + 0 = 0$	Sum of 0 with a carry of 0
$0 + 1 = 1$	Sum of 1 with a carry of 0
$1 + 0 = 1$	Sum of 1 with a carry of 0
$1 + 1 = 10$	Sum of 0 with a carry of 1



Carry bits

1	$+ 0 + 0 = 01$	Sum of 1 with a carry of 0
1	$+ 1 + 0 = 10$	Sum of 0 with a carry of 1
1	$+ 0 + 1 = 10$	Sum of 0 with a carry of 1
1	$+ 1 + 1 = 11$	Sum of 1 with a carry of 1

In binary $1 + 1 = 10$, not 2.

Binary Addition

EXAMPLE 2-7

Add the following binary numbers:

- (a) $11 + 11$ (b) $100 + 10$ (c) $111 + 11$ (d) $110 + 100$

Solution

The equivalent decimal addition is also shown for reference.

(a) $11 + 11$	(b) $100 + 10$	(c) $111 + 11$	(d) $110 + 100$
11 + 11 110 3 + 3 6	100 + 10 110 4 + 2 6	111 + 11 1010 7 + 3 10	110 + 100 1010 6 + 4 10



Related Problem

Add 1111 and 1100.

Binary Subtraction

- The four basic rules for subtracting bits are as follows:

$$\begin{array}{l} 0 - 0 = 0 \\ 1 - 1 = 0 \\ 1 - 0 = 1 \\ 10 - 1 = 1 \quad 0 - 1 \text{ with a borrow of } 1 \end{array}$$

- When subtracting numbers, you sometimes have to borrow from the next column to the left. A borrow is required in binary only when you try to subtract a 1 from a 0. In this case, when a 1 is borrowed from the next column to the left, a 10 is created in the column being subtracted, and the last of the four basic rules just listed must be applied.

In binary $10 - 1 = 1$, not 9.

Binary Subtraction

EXAMPLE 2–8

Perform the following binary subtractions: (a) $11 - 01$ (b) $11 - 10$

Solution

(a) $11 - 01$

Binary subtraction

$$\begin{array}{r} 11 \\ - 01 \\ \hline 10 \end{array}$$

Equivalent decimal subtraction

$$\begin{array}{r} 3 \\ - 1 \\ \hline 2 \end{array}$$

(b) $11 - 10$

Binary subtraction

$$\begin{array}{r} 11 \\ - 10 \\ \hline 01 \end{array}$$

Equivalent decimal subtraction

$$\begin{array}{r} 3 \\ - 2 \\ \hline 1 \end{array}$$

i No borrows were required in this example. The binary number 01 is the same as 1.



Related Problem

Subtract 100 from 111.

Binary Subtraction

EXAMPLE 2–9

Subtract 011 from 101.

Solution

Let’s examine exactly what was done to subtract the two binary numbers since a borrow is required. Begin with the right column.

	Left (2 ²)	Middle (2 ¹)	Right (2 ⁰)
101	1	0	1
– 011	0	1	1
010	0	1	0

1 Right column (2⁰)

$$1 - 1 = 0.$$

Write 0 in the rightmost position.

2 Middle column (2¹)

The top digit is 0, but we need to subtract 1, so we borrow 1 from the next column to the left, making a 10 in this column. Then $10 - 1 = 1$.

Write 1 in the middle position.

3 Left column (2²)

After the borrow, the left digit becomes 0. Then $0 - 0 = 0$.

Write 0 in the leftmost position.

$$101 - 011 = \mathbf{010}$$



Related Problem

Subtract 101 from 110.

Binary Multiplication

- The four basic rules for multiplying bits are as follows:

$$0 \times 0 = 0$$

$$0 \times 1 = 0$$

$$1 \times 0 = 0$$

$$1 \times 1 = 1$$

- Multiplication is performed with binary numbers in the same manner as with decimal numbers. It involves forming partial products, shifting each successive partial product left one place, and then adding all the partial products.

Binary Multiplication

EXAMPLE 2–10 Perform the following binary multiplications: (a) 11×11 (b) 101×111

Solution

(a) 11×11

Partial products {
$$\begin{array}{r} 11 \\ \times 11 \\ \hline 11 \\ +11 \\ \hline 1001 \end{array}$$

($11 \times 1 = 11$)
($11 \times 1 = 11$, shift left by 1)

Equivalent decimal check

$$\begin{aligned} 11_2 &= 3 \\ 11_2 &= 3 \\ 3 \times 3 &= 9 \\ 9_{10} &= 1001_2 \end{aligned}$$

(b) 101×111

Partial products {
$$\begin{array}{r} 101 \\ \times 111 \\ \hline 101 \\ 000 \\ +101 \\ \hline 100011 \end{array}$$

($101 \times 1 = 101$)
($101 \times 1 = 101$, shift left by 1)
($101 \times 1 = 101$, shift left by 2)

Equivalent decimal check

$$\begin{aligned} 101_2 &= 5 \\ 111_2 &= 7 \\ 5 \times 7 &= 35 \\ 35_{10} &= 100011_2 \end{aligned}$$



Related Problem

Multiply 1101×1010 .

Binary Division

EXAMPLE 2-11 Perform the following binary divisions: (a) $110 \div 11$ (b) $110 \div 10$

Solution

(a) $110 \div 11$

$$\begin{array}{r} 10 \\ 11 \overline{) 110} \\ \underline{11} \\ 000 \end{array}$$

Steps:

- 1 11 goes into 11 once.
Write 1 in the quotient,
subtract $11 - 11 = 0$.
- 2 Bring down the next 0.
11 does not go into 0.
Write 0 in the quotient.

Equivalent decimal check:

$$\begin{aligned} 110_2 &= 6 \\ 11_2 &= 3 \\ 6 \div 3 &= 2 \\ 2_{10} &= 10_2 \end{aligned}$$

Quotient: **10**
Remainder: **0**

(b) $110 \div 10$

$$\begin{array}{r} 11 \\ 10 \overline{) 110} \\ \underline{10} \\ 10 \\ \underline{10} \\ 00 \end{array}$$

Steps:

- 1 10 goes into 11 once.
Write 1 in the quotient,
subtract $11 - 10 = 1$.
- 2 Bring down the next 0.
10 goes into 10 once.
Write 1 in the quotient,
subtract $10 - 10 = 0$.
- 3 No more digits to bring down.
Remainder is 0.

Equivalent decimal check:

$$\begin{aligned} 110_2 &= 6 \\ 10_2 &= 2 \\ 6 \div 2 &= 3 \\ 3_{10} &= 11_2 \end{aligned}$$

Quotient: **11**
Remainder: **0**



Related Problem Divide 1100 by 100.

5. Complements of Binary Numbers

Finding the 1's Complement

- The 1's complement of a binary number is found by changing all 1s to 0s and all 0s to 1s, as illustrated below:

1	0	1	1	0	0	1	0
Binary number							
↓	↓	↓	↓	↓	↓	↓	↓
0	1	0	0	1	1	0	1
1's complement							

- The simplest way to obtain the 1's complement of a binary number with a digital circuit is to use parallel inverters (NOT circuits)

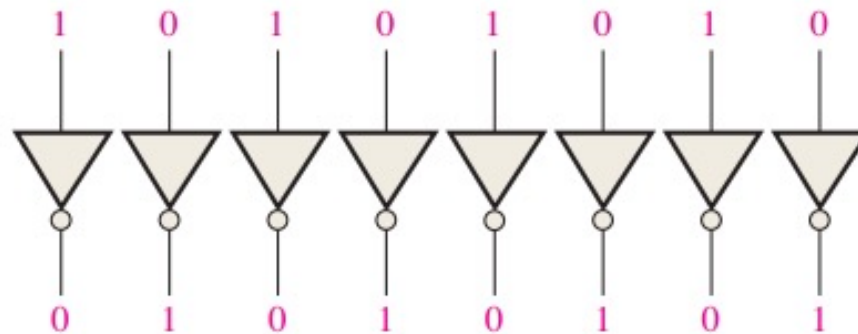


FIGURE 2-2 Example of inverters used to obtain the 1's complement of a binary number.

Finding the 2's Complement

The 2's complement of a binary number is found by adding 1 to the LSB of the 1's complement.

$$\text{2's complement} = (\text{1's complement}) + 1$$

EXAMPLE 2-12

Find the 2's complement of 10110010.

Solution

10110010	Binary number
01001101	1's complement
+ 1	Add 1
01001110	2's complement

Related Problem

Determine the 2's complement of 11001011.

Finding the 2's Complement

An alternative method of finding the 2's complement of a binary number is as follows:

- Start at the right with the LSB and write the bits as they are up to and including the first 1.
- Take the 1's complements of the remaining bits.

EXAMPLE 2-13

Find the 2's complement of 10111000 using the alternative method.

Solution

10111000

Binary number

01001000

2's complement

1's complements
of original bits

These bits stay the same.



Related Problem

Find the 2's complement of 11000000.

Finding the 2's Complement

- The 2's complement of a negative binary number can be realized using inverters and an adder.
- This illustrates how an 8-bit number can be converted to its 2's complement by first inverting each bit (taking the 1's complement) and then adding 1 to the 1's complement with an adder circuit.

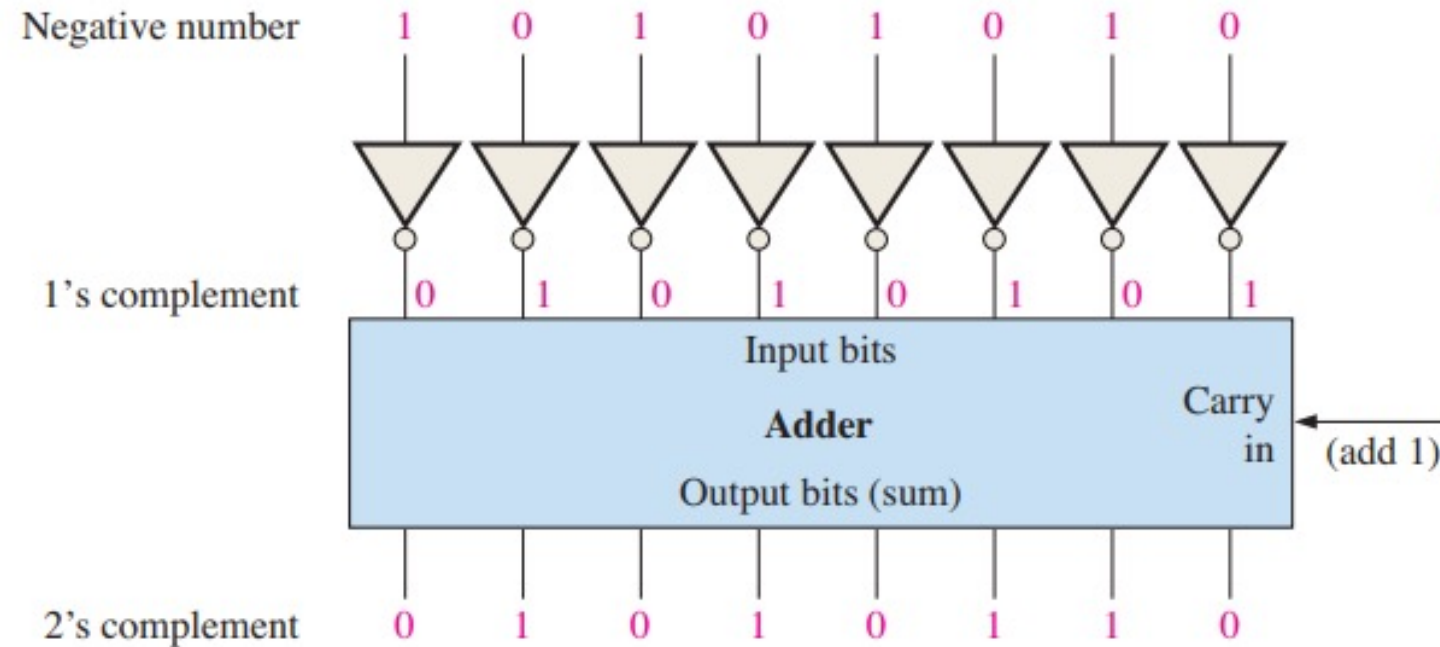


FIGURE 2-3 Example of obtaining the 2's complement of a negative binary number.

6. Signed Numbers

- Digital systems, such as the computer, must be able to handle both positive and negative numbers.
- A signed binary number consists of both sign and magnitude information.

The Sign Bit

The left-most bit in a signed binary number is the **sign bit**, which tells you whether the number is positive or negative.

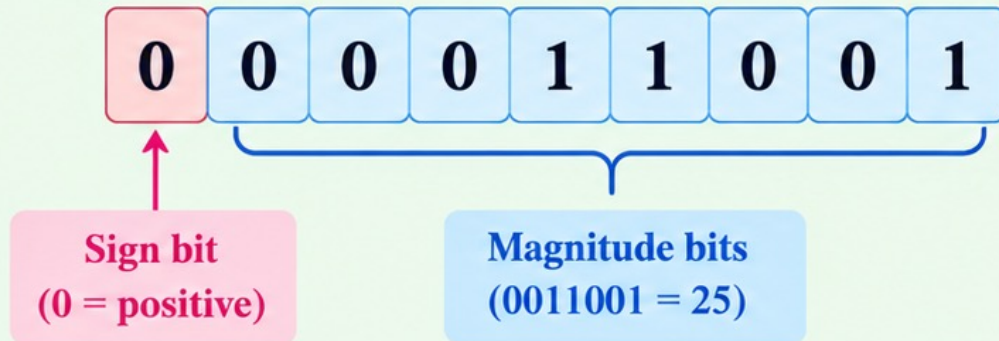
A 0 sign bit indicates a positive number, and a 1 sign bit indicates a negative number.

Sign-Magnitude Form

When a signed binary number is represented in sign-magnitude, the left-most bit is the **sign bit** and the remaining bits are the **magnitude bits**.

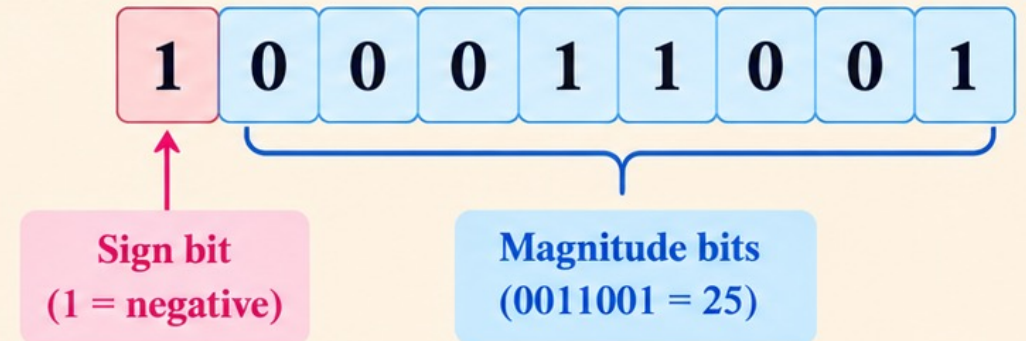
Example: +25

The decimal number +25 is expressed as an 8-bit signed binary number using the sign-magnitude form as



Example: -25

The decimal number -25 is expressed as an 8-bit signed binary number using the sign-magnitude form as



In the sign-magnitude form, a negative number has the same magnitude bits as the corresponding positive number but the sign bit is a 1 rather than a zero.

1's Complement Form

- Positive numbers in 1's complement form are represented the same way as the positive sign-magnitude numbers.
- Negative numbers, however, are the 1's complements of the corresponding positive numbers.
For example, using eight bits, the decimal number -25 is expressed as the 1's complement of $+25$ (00011001) as 11100110.

Example: +25 (8 bits)

The decimal number $+25$ is represented in 1's complement form as (same as sign-magnitude for positive numbers).



Sign bit
(0 = positive)

Magnitude bits
(0011001 = 25)

Example: -25 (8 bits)

The decimal number -25 is the 1's complement of $+25$.
Complement each bit of 00011001 to get



Sign bit
(1 = negative)

Magnitude bits
(complement of 0011001)



In the 1's complement form, a negative number is the 1's complement of the corresponding positive number.

2's Complement Form

- Positive numbers in 2's complement form are represented the same way as in the sign magnitude and 1's complement forms.
- Negative numbers are the 2's complements of the corresponding positive numbers. Again, using eight bits, let's take decimal number -25 and express it as the 2's complement of $+25$ (00011001).
- Inverting each bit and adding 1, you get $-25 = 11100111$.

1 Write the positive number

$+25$ in 8-bit binary form:

0	0	0	1	1	0	0	1
---	---	---	---	---	---	---	---

$+25 = 00011001$

2 Invert each bit (1's complement)

Change $0 \leftrightarrow 1$:

1	1	1	0	0	1	1	0
---	---	---	---	---	---	---	---

3 Add 1

	1	1	1	0	0	1	1	0
+	0	0	0	0	0	0	0	1
	1	1	1	0	0	1	1	1

$-25 = 11100111$



In the 2's complement form, a negative number is the 2's complement of the corresponding positive number.

EXAMPLE 2-14

Express the decimal number -39 as an 8-bit number in the sign-magnitude, 1's complement, and 2's complement forms.

Solution

(a) Sign-magnitude form

First, write the 8-bit number for $+39$.

00100111 (+39)

- 1 Change the sign bit to 1 and leave the magnitude bits as they are.

10100111 (-39)

↑
Sign bit
(1 = negative)

└───┘
Magnitude bits
(0100111 = 39)

(b) 1's complement form

Take the 1's complement of $+39$ (00100111) by inverting each bit.

- 1 Invert each bit ($0 \leftrightarrow 1$).

00100111 (+39)

11011000 (1's complement)

(c) 2's complement form

Take the 2's complement of $+39$ (00100111) by adding 1 to the 1's complement.

- 1 Start with the 1's complement.

11011000 (1's complement)

- 2 Add 1.

11011000
+ 1
┌───┘
11011001 (2's complement)



Related Problem Express $+19$ and -19 as 8-bit numbers in sign-magnitude, 1's complement, and 2's complement.

The Decimal Value of Signed Numbers

Sign-magnitude

- Decimal values of positive and negative numbers in the sign-magnitude form are determined by summing the weights in all the magnitude bit positions where there are 1s and ignoring those positions where there are zeros.
- The sign is determined by examination of the sign bit.

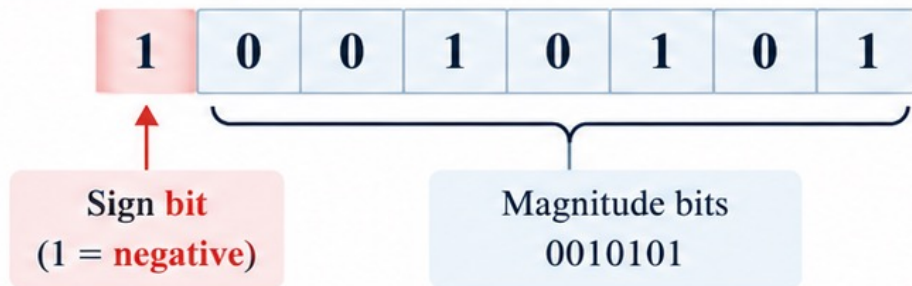
EXAMPLE 2-15

Determine the decimal value of this signed binary number expressed in sign-magnitude: 10010101.

Solution

1 Identify the sign bit and magnitude bits

The left-most bit is the sign bit, and the remaining 7 bits are the magnitude bits.



2 Find the value of the magnitude bits

The seven magnitude bits and their powers-of-two weights are as follows:

2^6	2^5	2^4	2^3	2^2	2^1	2^0
0	0	1	0	1	0	1

3 Sum the weights where there are 1s

The value of the magnitude bits is

$$16 + 4 + 1 = 21$$

Since the sign bit is 1, the number is negative.

$$\mathbf{10010101 = -21}$$



Related Problem Determine the decimal value of the sign-magnitude number 01110111.

1's Complement

- Decimal values of positive numbers in the 1's complement form are determined by summing the weights in all bit positions where there are 1s and ignoring those positions where there are zeros.
- Decimal values of negative numbers are determined by assigning a negative value to the weight of the sign bit, summing all the weights where there are 1s, and adding 1 to the result.

EXAMPLE 2–16

Determine the decimal values of the signed binary numbers expressed in 1's complement:

(a) 00010111 (b) 11101000

Solution

(a) 00010111

The bits and their powers-of-two weights for the positive number are as follows:

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
0	0	0	1	0	1	1	1

Summing the weights where there are 1s,

$$16 + 4 + 2 + 1 = +23$$

$$\mathbf{00010111 = +23}$$

(b) 11101000

The bits and their powers-of-two weights for the negative number are as follows. Notice that the negative sign bit has a weight of -2^7 or -128 .

-2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
1	1	1	0	1	0	0	0

Summing the weights where there are 1s,

$$-128 + 64 + 32 + 8 = -24$$

Adding 1 to the result, the final decimal number is

$$-24 + 1 = -23$$

$$\mathbf{11101000 = -23}$$



Related Problem Determine the decimal value of the 1's complement number 11101011.

2's Complement

- Decimal values of positive and negative numbers in the 2's complement form are determined by summing the weights in all bit positions where there are 1s and ignoring those positions where there are zeros.
- The weight of the sign bit in a negative number is given a negative value.

EXAMPLE 2–17

Determine the decimal values of the signed binary numbers expressed in 2's complement:

(a) 01010110 (b) 10101010

Solution

(a) 01010110

- 1 Sign bit is 0 $\Rightarrow$ positive number.
- 2 Write the bit weights and sum the 1s.

-2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
0	1	0	1	0	1	1	0

$$64 + 16 + 4 + 2 = 86$$

$$01010110 = +86$$

(b) 10101010

- 1 Sign bit is 1 $\Rightarrow$ negative number.
- 2 Write the bit weights and sum the 1s.

-2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
1	0	1	0	1	0	1	0

$$-128 + 32 + 8 + 2 = -86$$

$$10101010 = -86$$



Related Problem Determine the decimal value of the 2's complement number 11010111.

Range of Signed Integer Numbers

1 Number of combinations

- We have used 8-bit numbers for illustration because the 8-bit grouping is common in most computers and has been given the special name byte. With one byte or eight bits, you can represent 256 different numbers. With two bytes or sixteen bits, you can represent 65,536 different numbers. The formula for finding the number of different combinations of n bits is

$$\text{Total combinations} = 2^n$$

2 Range of 2's complement signed numbers

- For 2's complement signed numbers, the range of values for n -bit numbers is

$$\text{Range} = -(2^{n-1}) \text{ to } +(2^{n-1} - 1)$$



Note The range of magnitude values represented by binary numbers depends on the number of bits (n).

Floating-Point Numbers

- The floating-point number system, based on scientific notation, is capable of representing very large and very small numbers without an increase in the number of bits and also for representing numbers that have both integer and fractional components.
- A floating-point number (also known as a real number) consists of two parts plus a sign. The **mantissa** is the part of a floating-point number that represents the magnitude of the number and is between 0 and 1. The **exponent** is the part of a floating-point number that represents the number of places that the decimal point (or binary point) is to be moved.

Single-Precision Floating-Point Binary Numbers

1 Format



S : Sign bit (left-most bit)

E : Exponent (next 8 bits)

F : Mantissa / fraction (remaining 23 bits)

2 Value Formula

A normalized single-precision floating-point number has the value

$$\text{Number} = (-1)^S (1 + F)(2^{E-127})$$

S : 0 = positive, 1 = negative

E : biased exponent (subtract 127)

F : fractional part (binary fraction)

3 Example

Find the value of the floating-point number:

S	E	F
1	10010001	100011100010000000000000

Solution:

1 $S = 1 \Rightarrow$ negative

2 $E = 10010001_2 = 145 \Rightarrow$ exponent = $145 - 127 = 18$

3 $F = 0.100011100010000000000000_2 \Rightarrow 1 + F = 1.10001110001_2$

$$\begin{aligned}\text{Number} &= (-1)^1 (1.10001110001_2)(2^{18}) \\ &= -1100011100010000000000\end{aligned}$$

Single-Precision Floating-Point Binary Numbers

EXAMPLE 2-18

Convert the decimal number 3.248×10^4 to a single-precision floating-point binary number.

Solution

1 Convert to binary

$$\begin{aligned} 3.248 \times 10^4 &= 32480 = 111110111000000_2 \\ &= 1.11110111000000 \times 2^{14} \end{aligned}$$

2 Determine the fields

- 1 Sign bit: $S = 0$ (positive)
- 2 Exponent: $14 + 127 = 141 = 10001101_2$
- 3 Mantissa: fractional part after the leading 1
 $F = 11110111000000000000000$ (23 bits)

3 Form the floating-point number

<i>S</i> 1 bit	Exponent (<i>E</i>) 8 bits	Mantissa (<i>F</i>) 23 bits
0	10001101	11110111000000000000000

4 Result

0 10001101 11110111000000000000000



Related Problem

Determine the binary value of the following floating-point binary number:

0 10011000 10000100010100110000000

7. Arithmetic Operations with Signed Numbers

Addition

The two numbers in an addition are the addend and the augend. The result is the sum.

There are four cases that can occur when two signed binary numbers are added.

- Both numbers positive
- Positive number with magnitude larger than negative number
- Negative number with magnitude larger than positive number
- Both numbers negative

Both numbers positive:

00000111	7
+ 00000100	+ 4
-----	-----
00001011	11

The sum is positive and is therefore in true (uncomplemented) binary.

Addition of two positive numbers yields a positive number.

Positive number with magnitude larger than negative number:

	00001111	15
	+ 11111010	+ -6
	-----	-----
Discard carry → 1	00001001	9

The final carry bit is discarded. The sum is positive and therefore in true (uncomplemented) binary.

Addition of a positive number and a smaller negative number yields a positive number.

Negative number with magnitude larger than positive number:

$$\begin{array}{r} 00010000 \\ + 11101000 \\ \hline 11111000 \end{array} \qquad \begin{array}{r} 16 \\ + -24 \\ \hline -8 \end{array}$$

The sum is negative and therefore in 2's complement form.

Addition of a positive number and a larger negative number or two negative numbers yields a negative number in 2's complement.

Both numbers negative:

$$\begin{array}{r} 11111011 \\ + 11110111 \\ \hline 11110010 \end{array} \qquad \begin{array}{r} -5 \\ + -9 \\ \hline -14 \end{array}$$

Discard carry $\longrightarrow$ 1

The final carry bit is discarded. The sum is negative and therefore in 2's complement form.

1

Overflow Condition

- When two numbers are added and the number of bits required to represent the sum exceeds the number of bits in the two numbers, an overflow results (indicated by an incorrect sign bit).
- Overflow can occur only when both numbers are positive or both numbers are negative. If the sign bit of the result is different from the sign bits of the numbers being added, overflow is indicated.

Example

$$\begin{array}{r}
 01111101 \quad (125) \\
 + 00111010 \quad (58) \\
 \hline
 10110111 \quad (183)
 \end{array}$$

Sign bit incorrect $\rightarrow$ (points to the first bit of the result, 1)
 Magnitude incorrect $\rightarrow$ (points to the remaining bits of the result, 0110111)

2

Numbers Added Two at a Time

- To add a string of numbers, add two at a time: add the first two numbers, then add the third number to the sum, then add the fourth number, and so on. This is how computers add strings of numbers.

Numbers Added Two at a Time

EXAMPLE 2-19

Add the signed numbers: 01000100, 00011011, 00001110, and 00010010.

Solution

Add the numbers two at a time.

Step	Decimal	Binary (8 bits)	Note
1	$\begin{array}{r} 68 \\ + 27 \\ \hline 95 \end{array}$	$\begin{array}{r} 01000100 \\ + 00011011 \\ \hline 01011111 \end{array}$	Add 1st two numbers
2	$\begin{array}{r} 95 \\ + 14 \\ \hline 109 \end{array}$	$\begin{array}{r} 01011111 \\ + 00001110 \\ \hline 01101101 \end{array}$	Add 3rd number
3	$\begin{array}{r} 109 \\ + 18 \\ \hline 127 \end{array}$	$\begin{array}{r} 01101101 \\ + 00010010 \\ \hline \mathbf{01111111} \end{array}$	Add 4th number (Final sum)



Related Problem

Add 00110011, 10111111, and 01100011. These are signed numbers.

Subtraction

1 Basic Idea

- Subtraction is a special case of addition.
 $A - B$ is equivalent to $A + (-B)$.
- The result of a subtraction is called the difference.

3 Sign Change with 2's Complement

The sign of a positive or negative binary number is changed by taking its 2's complement.

2 Using 2's Complement

To subtract two signed numbers ($A - B$):

- 1 Take the 2's complement of B .
- 2 Add it to A .
- 3 Discard any final carry bit.



Related Example

Perform each of the following subtractions of the signed numbers:

(a) $00001000 - 00000011$

(b) $00001100 - 11110111$

(c) $11100111 - 00010011$

(d) $10001000 - 11100010$

EXAMPLE 2-20

Perform each of the following subtractions of the signed numbers:

(a) $00001000 - 00000011$

(b) $00001100 - 11110111$

(c) $11100111 - 00010011$

(d) $10001000 - 11100010$

SolutionUse 2's complement: $A - B = A + (2\text{'s complement of } B)$.

(a) $00001000 - 00000011$

$8 - 3 = 5$

00001000	Minuend (+8)
+ 11111101	2's complement of 00000011 (-3)

Discard carry $\rightarrow$ **00000101** Difference (+5)

(b) $00001100 - 11110111$

$12 - (-9) = 21$

00001100	Minuend (+12)
+ 00001001	2's complement of 11110111 (+9)

00010101 Difference (+21)

(c) $11100111 - 00010011$

$-25 - (+19) = -44$

11100111	Minuend (-25)
+ 11101101	2's complement of 00010011 (-19)

Discard carry $\rightarrow$ **11010100** Difference (-44)

(d) $10001000 - 11100010$

$-120 - (-30) = -90$

10001000	Minuend (-120)
+ 00011110	2's complement of 11100010 (+30)

10100110 Difference (-90)

**Related Problem**

Subtract 01000111 from 01011000.

Multiplication

- The numbers in a multiplication are the multiplicand, the multiplier, and the product.
- These are illustrated in the following decimal multiplication:

8	Multiplicand
$\times 3$	Multiplier
24	Product
- Direct addition and partial products are two basic methods for performing multiplication using addition.

The direct addition method

- In the direct addition method, you add the multiplicand a number of times equal to the multiplier

The direct addition method

EXAMPLE 2-21

Multiply the signed binary numbers: 01001101 (multiplicand) and 00000100 (multiplier) using the direct addition method.

1 Check the numbers

- Both numbers are positive (sign bit = 0) → use direct addition.
- Multiplier 00000100 = 4 → add the multiplicand four times.

2 Repeated addition

Step	Operation	Result (8 bits)	
1	01001101	01001101	1st time
2	+ 01001101	10011010	2nd time (partial sum)
3	+ 01001101	11100111	3rd time (partial sum)
4	+ 01001101	100110100	4th time (product)

3 Result

Product = 100110100 (8 bits)

- The multiplicand is positive, so all bits in the product are magnitude bits.



Related Problem

Multiply 01100001 by 00000110 using the direct addition method.

The Partial Products Method

1 Idea

- Multiply the multiplicand by each multiplier bit (from right to left), shift each partial product left, and add them.

2 Steps

- 1. Multiply multiplicand by each multiplier bit.
- 2. Shift each partial product left.
- 3. Add all partial products.

3 Example (decimal)

- 239×123 :
Partial products: 717, 478, 239
Final product: 29,397

4 Example (binary)

Multiply 01001101 (multiplicand) by 00000100 (multiplier).

01001101	(multiplicand)
$\times$ 00000100	(multiplier)
00000000	($\times 0$, shift 0)
00000000	($\times 0$, shift 1)
00000000	($\times 0$, shift 2)
+ 01001101	($\times 1$, shift 3)
001001101000	(product, 12 bits)

- The multiplier is 00000100 (4), so only the 3rd bit is 1.
Product = 001001101000 (decimal 316).

Signed Binary Multiplication (Partial Products Method)

1 Determine the Sign

- Same signs $\rightarrow$ product is positive.
- Different signs $\rightarrow$ product is negative.

2 Convert to True Form

- If a number is negative, convert it from 2's complement to its true (unsigned) form.
- Use the absolute values for multiplication.

3 Generate Partial Products

- Start from the least significant bit (LSB) of the multiplier.
- If the bit is 1 $\rightarrow$ use the multiplicand.
If the bit is 0 $\rightarrow$ use 0.
- Shift each partial product left by 1 bit.

4 Add Partial Products

- Add all partial products to get the final product (magnitude).

5 Apply the Sign

- If the product is negative, take the 2's complement.
- If positive, keep the result in true form.
- Attach the sign bit to the product.



Summary

Use the signs to determine the sign, multiply the absolute values using partial products, then apply the sign to get the final product.

EXAMPLE 2-22

Multiply the signed binary numbers: 01010011 (multiplicand) and 11000101 (multiplier).

1 Find the sign

- Multiplicand sign = 0 (positive)
- Multiplier sign = 1 (negative)
→ Product will be negative (sign = 1).

2 Convert to true form

- Multiplier is negative → take 2's complement.

11000101 → 00111011

5 Apply the sign

- Product is negative → take 2's complement and attach sign bit.

10011001000010 → 011001101111

Attach sign bit 1 → **1** 011001101111 (final product, 15 bits)

3 Generate partial products

- Use the magnitude bits.
- Start from LSB, add the multiplicand when the bit is 1, shift left each time.

4 Add partial products

- Add all partial products to get the product (magnitude).

3 & 4 Multiplication (magnitude)

Operation	Note
01010011	Multiplicand (83)
× 00111011	Multiplier (59)
01010011	bit 0 = 1 (shift 0)
+ 01010011	bit 1 = 1 (shift 1)
10100110	sum
+ 00000000	bit 2 = 0 (shift 2)
10100110	sum
+ 10100110	bit 3 = 1 (shift 3)
1000110010	sum
+ 10100010	bit 4 = 1 (shift 4)
10011001000010	sum
+ 00000000	bit 5 = 0 (shift 5)
10011001000010	sum
+ 00000000	bit 6 = 0 (shift 6)
10011001000010	Product (magnitude)



Related Problem

Verify the multiplication is correct by converting to decimal numbers and performing the multiplication.

Division

1 Terminology

- **Dividend:** the number being divided.
- **Divisor:** the number used to divide.
- **Quotient:** the result of the division (number of times the divisor goes into the dividend).

2 Division format

$$\frac{\text{dividend}}{\text{divisor}} = \text{quotient}$$

3 Idea

- The divisor is subtracted from the dividend a number of times equal to the quotient.
- The final result is the quotient and a remainder.

4 Example (decimal)

Divide 21 by 7.

21	Dividend
$- 7$	1st subtraction of divisor
14	1st partial remainder
$- 7$	2nd subtraction of divisor
7	2nd partial remainder
$- 7$	3rd subtraction of divisor
0	Zero remainder

Quotient = 3, Remainder = 0
(because $21 = 7 \times 3$)

Signed Binary Division

1 Sign of the Quotient

- Same signs $\rightarrow$ quotient is positive.
- Different signs $\rightarrow$ quotient is negative.

2 Preparation

- Convert both dividend and divisor to true (unsigned) form (use 2's complement if negative).
- Initialize the quotient to 0.

3 Division Steps

- 1 Determine the sign of the quotient and initialize the quotient to 0.
- 2 Subtract the divisor from the dividend (using 2's complement).
If the remainder is positive, add 1 to the quotient and continue.
- 3 Repeat the subtraction with the new remainder and add 1 to the quotient.
Stop when the remainder is zero or negative.



Summary

Use the signs to determine the sign of the quotient, convert to true form, repeatedly subtract the divisor and count the number of times, then apply the sign.

EXAMPLE 2-23

Divide 01100100 by 00011001.

1 Determine the sign

- Both numbers are positive → quotient is positive.
- Initialize quotient: 00000000.

2 1st subtraction

$$\begin{array}{r} 01100100 \quad (\text{dividend}) \\ + 11100111 \quad (2\text{'s complement of divisor}) \\ \hline 01001011 \quad (1\text{st remainder, positive}) \end{array}$$

Add 1 to quotient:
00000000 + 00000001
= 00000001

3 2nd subtraction

$$\begin{array}{r} 01001011 \quad (1\text{st remainder}) \\ + 11100111 \quad (2\text{'s complement of divisor}) \\ \hline 00110010 \quad (2\text{nd remainder, positive}) \end{array}$$

Add 1 to quotient:
00000001 + 00000001
= 00000010

4 Continue

Repeat the subtraction and add 1 to the quotient until the remainder is 0 or negative.

5 Result

Quotient = 00000111 (7)

Remainder = 00000001 (1)

**Related Problem**

Verify the result by converting to decimal numbers.

Step 4: Subtract the divisor from the 2nd partial remainder using 2's complement addition.

00110010	2nd partial remainder
+ 11100111	2's complement of divisor
00011001	Positive 3rd partial remainder

Add 1 to quotient: $00000010 + 00000001 = 00000011$.

Step 5: Subtract the divisor from the 3rd partial remainder using 2's complement addition.

00011001	3rd partial remainder
+ 11100111	2's complement of divisor
00000000	Zero remainder

Add 1 to quotient: $00000011 + 00000001 = \mathbf{00000100}$ (final quotient). The process is complete.

Related Problem

Verify that the process is correct by converting to decimal numbers and performing the division.

8. Hexadecimal Numbers

- The hexadecimal number system has sixteen characters; it is used primarily as a compact way of displaying or writing binary numbers because it is very easy to convert between binary and hexadecimal.
- The hexadecimal number system has a base of sixteen; that is, it is composed of 16 numeric and alphabetic characters.
- Most digital systems process binary data in groups that are multiples of four bits, making the hexadecimal number very convenient because each hexadecimal digit represents a 4-bit binary number

TABLE 2-3

Decimal	Binary	Hexadecimal
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

Binary-to-Hexadecimal Conversion

Group the binary number into 4-bit blocks from right to left and replace each block with its hexadecimal symbol.

EXAMPLE 2-24 (a) Convert 1100101001010111 to hexadecimal.

1100	1010	0101	0111
⏟	⏟	⏟	⏟
↓	↓	↓	↓
C	A	5	7

= CA57₁₆

EXAMPLE 2-24 (b) Convert 11111000101101001 to hexadecimal.

0011	1111	0001	0110	1001
⏟	⏟	⏟	⏟	⏟
↓	↓	↓	↓	↓
3	F	1	6	9

(Two zeros added at the left to complete a 4-bit block.)

= 3F169₁₆



Related Problem

Convert the binary number 1001111011110011100 to hexadecimal.

Hexadecimal-to-Binary Conversion

Replace each hexadecimal digit with its 4-bit binary equivalent.

(a) $10A4_{16}$

1	0	A	4
↓	↓	↓	↓
0001	0000	1010	0100

$= 1000010100100$

The MSB has three leading zeros to form a 4-bit group.

(b) $CF8E_{16}$

C	F	8	E
↓	↓	↓	↓
1100	1111	1000	1110

$= 1100111110001110$

(c) 9742_{16}

9	7	4	2
↓	↓	↓	↓
1001	0111	0100	0010

$= 1001011101000010$



Related Problem

Convert the hexadecimal number 6BD3 to binary.

Hexadecimal-to-Decimal Conversion

Convert a hexadecimal number to binary, then to decimal.

EXAMPLE 2-26 (a)

Convert $1C_{16}$ to decimal.

$$\begin{array}{cc} 1 & C \\ \downarrow & \downarrow \\ 0001 & 1100 \\ \underbrace{\hspace{1cm}} & \underbrace{\hspace{1cm}} \\ 00011100 \end{array}$$

$$\begin{aligned} &2^4 + 2^3 + 2^2 \\ &= 16 + 8 + 4 \\ &= \mathbf{28}_{10} \end{aligned}$$

EXAMPLE 2-26 (b)

Convert $A85_{16}$ to decimal.

$$\begin{array}{ccc} A & 8 & 5 \\ \downarrow & \downarrow & \downarrow \\ 1010 & 1000 & 0101 \\ \underbrace{\hspace{1cm}} & \underbrace{\hspace{1cm}} & \underbrace{\hspace{1cm}} \\ 101010000101 \end{array}$$

$$\begin{aligned} &2^{11} + 2^9 + 2^7 + 2^2 + 2^0 \\ &= 2048 + 512 + 128 + 4 + 1 \\ &= \mathbf{2693}_{10} \end{aligned}$$



Related Problem

Convert the hexadecimal number 6BD to decimal.

Hexadecimal-to-Decimal Conversion

Multiply each hexadecimal digit by its weight (power of 16) and add the products.

Weights (right → left):
 $16^0, 16^1, 16^2, \dots$

EXAMPLE 2-27 (a)

Convert $E5_{16}$ to decimal.

E	5
↓	↓
14	5

$$\begin{aligned}E5_{16} &= (14 \times 16) + (5 \times 1) \\&= 224 + 5 \\&= \mathbf{229}_{10}\end{aligned}$$

EXAMPLE 2-27 (b)

Convert $B2F8_{16}$ to decimal.

B	2	F	8
↓	↓	↓	↓
11	2	15	8

$$\begin{aligned}B2F8_{16} &= (11 \times 16^3) + (2 \times 16^2) \\&\quad + (15 \times 16) + (8 \times 1) \\&= 45,056 + 512 + 240 + 8 \\&= \mathbf{45,816}_{10}\end{aligned}$$



Related Problem

Convert $60A_{16}$ to decimal.

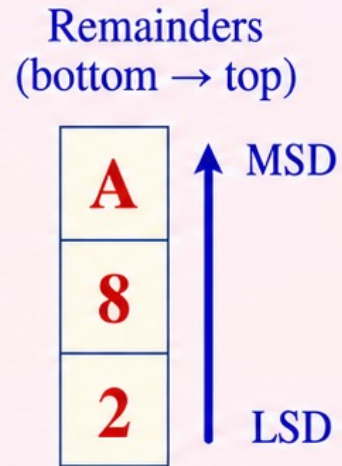
Decimal-to-Hexadecimal Conversion

Repeatedly divide the decimal number by 16. The remainders (from bottom to top) form the hexadecimal number.

EXAMPLE 2-28

Convert 650 to hexadecimal.

$$\begin{array}{l} 650 \div 16 = 40 \quad \text{remainder } 10 = \mathbf{A} \\ \downarrow \\ 40 \div 16 = 2 \quad \text{remainder } \mathbf{8} \\ \downarrow \\ 2 \div 16 = 0 \quad \text{remainder } \mathbf{2} \end{array}$$



$$= \mathbf{28A}_{16}$$



Note

- The first remainder is the least significant digit (LSD).
- Continue until the quotient is zero.



Related Problem

Convert 2591 to hexadecimal.

Hexadecimal Addition

Add hexadecimal numbers using these rules (A–F = 10–15):

- 1 Add each column (think in decimal).
e.g., $5_{16} = 5_{10}$, $C_{16} = 12_{10}$.
- 2 If the sum ≤ 15 , write the hex digit.
- 3 If the sum > 15 , write (sum $- 16$) and carry 1.

EXAMPLE

Add $5C_{16}$ and $A7_{16}$.

$$\begin{array}{r} \\ 5 \\ + A \\ \hline 1 3_{16} \end{array}$$

$$C + 7 = 12 + 7 = 19$$

→ write 3, carry 1

$$5 + A + 1 = 5 + 10 + 1 = 16$$

→ write 0, carry 1

→ final carry = 1



Related Problem

Add $3F_{16}$ and $8D_{16}$.

EXAMPLE 2-29

Add the following hexadecimal numbers.

(a)

$$\begin{array}{r} 23_{16} \\ + 16_{16} \\ \hline 39_{16} \end{array}$$

Right: $3 + 6 = 9$ Left: $2 + 1 = 3$ **(b)**

$$\begin{array}{r} 58_{16} \\ + 22_{16} \\ \hline 7A_{16} \end{array}$$

Right: $8 + 2 = 10 = A$ Left: $5 + 2 = 7$ **(c)**

$$\begin{array}{r} 2B_{16} \\ + 84_{16} \\ \hline AF_{16} \end{array}$$

Right: $B + 4 = 15 = F$ Left: $2 + 8 = 10 = A$ **(d)**

$$\begin{array}{r} DF_{16} \\ + AC_{16} \\ \hline 18B_{16} \end{array}$$

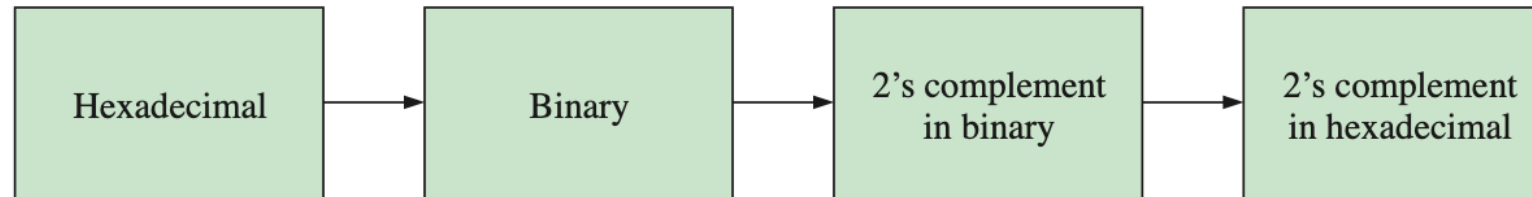
Right: $F + C = 27 = 11 \rightarrow B$ (carry 1)Left: $D + A + 1 = 24 = 8$ (carry 1)**Related Problem**Add $4C_{16}$ and $3A_{16}$.

Hexadecimal Subtraction

There are three ways to get the 2's complement of a hexadecimal number.

Method 1:

Convert the hexadecimal number to binary. Take the 2's complement of the binary number. Convert the result to hexadecimal.



Example:

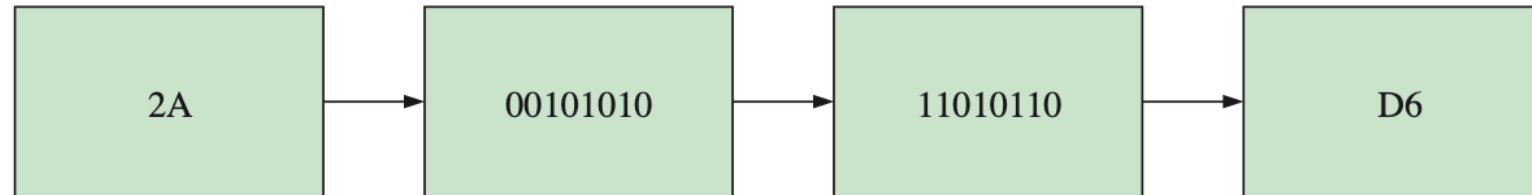
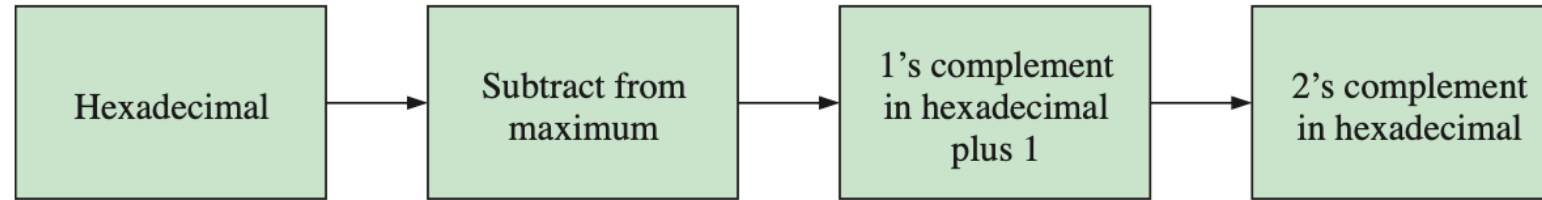


FIGURE 2-4 Getting the 2's complement of a hexadecimal number, Method 1.

Method 2:

Subtract the hexadecimal number from the maximum hexadecimal number and add 1.



Example:

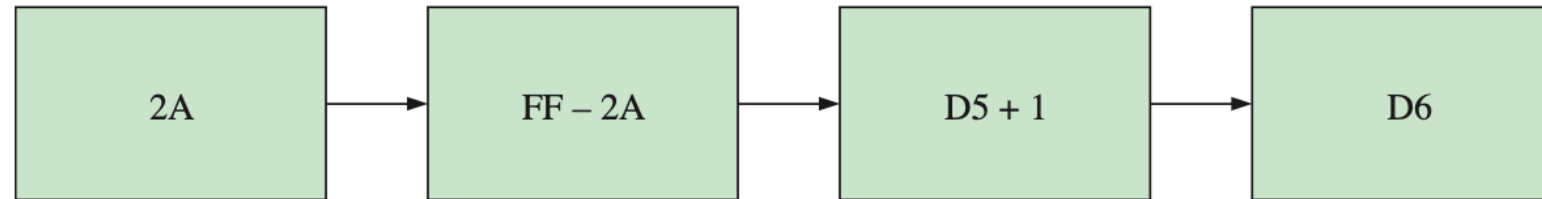


FIGURE 2-5 Getting the 2's complement of a hexadecimal number, Method 2.

Method 3:

Write the sequence of single hexadecimal digits. Write the sequence in reverse below the forward sequence. The 1's complement of each hex digit is the digit directly below it. Add 1 to the resulting number to get the 2's complement.

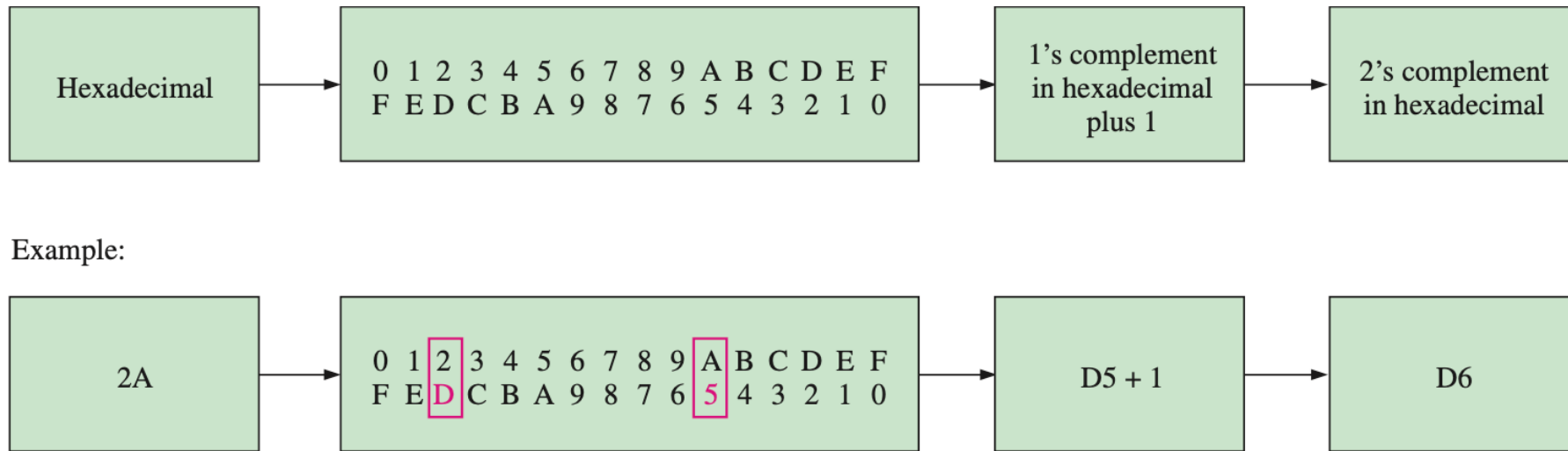


FIGURE 2-6 Getting the 2's complement of a hexadecimal number, Method 3.

Hexadecimal Subtraction (2's Complement)

Subtract by adding the 2's complement of the subtrahend (drop carry).

EXAMPLE 2-30 (a)

Subtract $84_{16} - 2A_{16}$.

$$\begin{aligned} 2A_{16} &= 00101010 \\ 2's \text{ comp} &= 11010110 \\ &= \mathbf{D6}_{16} \end{aligned}$$

$$\begin{array}{r} 84_{16} \\ + \mathbf{D6}_{16} \\ \hline \mathbf{15A}_{16} \end{array} \quad \begin{array}{l} \text{Add} \\ \text{(drop carry)} \end{array}$$

Result: $\mathbf{5A}_{16}$

EXAMPLE 2-30 (b)

Subtract $C3_{16} - 0B_{16}$.

$$\begin{aligned} 0B_{16} &= 00001011 \\ 2's \text{ comp} &= 11110101 \\ &= \mathbf{F5}_{16} \end{aligned}$$

$$\begin{array}{r} C3_{16} \\ + \mathbf{F5}_{16} \\ \hline \mathbf{1B8}_{16} \end{array} \quad \begin{array}{l} \text{Add} \\ \text{(drop carry)} \end{array}$$

Result: $\mathbf{B8}_{16}$



Related Problem

Subtract 173_{16} from BCD_{16} .

9. Octal Numbers

- The octal number system provides a convenient way to express binary numbers and codes
- The octal number system is composed of eight digits, which are

0, 1, 2, 3, 4, 5, 6, 7

- To count above 7, begin another column and start over:

10, 11, 12, 13, 14, 15, 16, 17, 20, 21, ...

- To distinguish octal numbers from decimal numbers or hexadecimal numbers, we will use the subscript 8 to indicate an octal number.

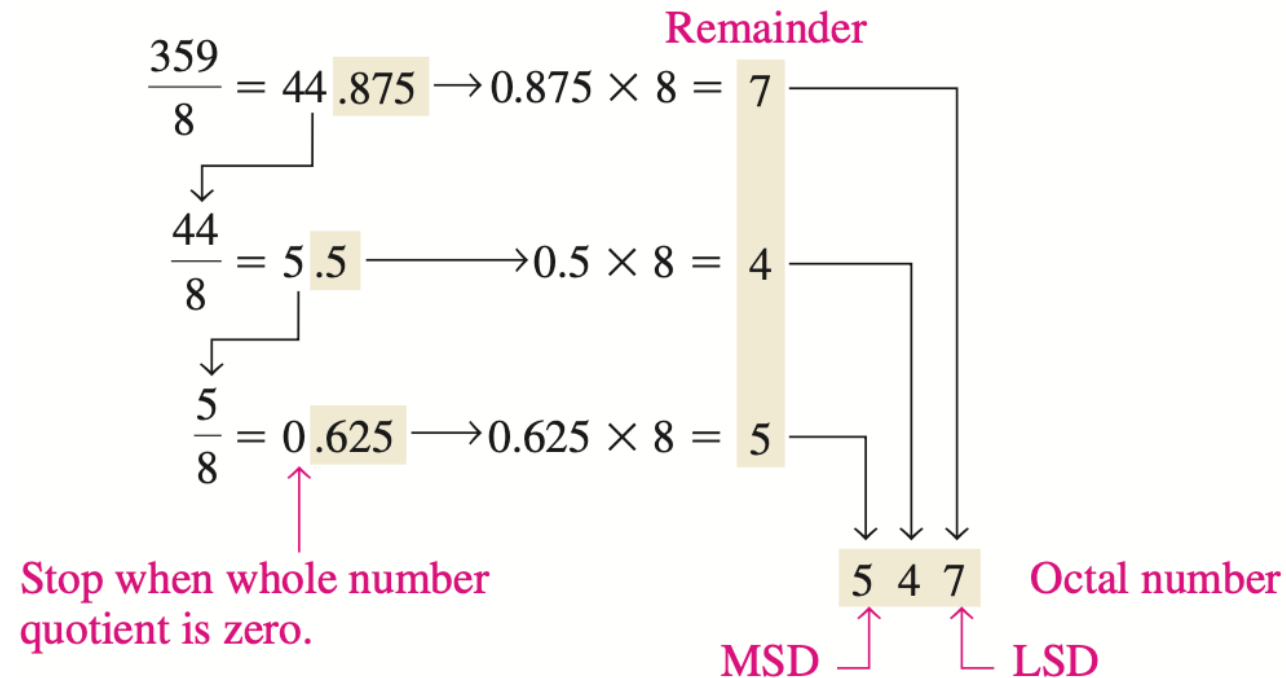
Octal-to-Decimal Conversion

- Since the octal number system has a base of eight, each successive digit position is an increasing power of eight, beginning in the right-most column with 8^0 .
- The evaluation of an octal number in terms of its decimal equivalent is accomplished by multiplying each digit by its weight and summing the products, as illustrated here for 2374_8 .

$$\begin{array}{rccccccc} & & & & \text{Weight:} & 8^3 & 8^2 & 8^1 & 8^0 \\ & & & & \text{Octal number:} & 2 & 3 & 7 & 4 \\ 2374_8 & = & (2 \times 8^3) & + & (3 \times 8^2) & + & (7 \times 8^1) & + & (4 \times 8^0) \\ & = & (2 \times 512) & + & (3 \times 64) & + & (7 \times 8) & + & (4 \times 1) \\ & = & 1024 & + & 192 & + & 56 & + & 4 & = 1276_{10} \end{array}$$

Decimal-to-Octal Conversion

- A method of converting a decimal number to an octal number is the repeated division- by-8 method, which is similar to the method used in the conversion of decimal numbers to binary or to hexadecimal.
- To show how it works, let's convert the decimal number 359 to octal. Each successive division by 8 yields a remainder that becomes a digit in the equivalent octal number. The first remainder generated is the least significant digit (LSD).



Octal-to-Binary Conversion

- Because each octal digit can be represented by a 3-bit binary number, it is very easy to convert from octal to binary. Each octal digit is represented by three bits:

TABLE 2-4

Octal/binary conversion.

Octal Digit	0	1	2	3	4	5	6	7
Binary	000	001	010	011	100	101	110	111

- To convert an octal number to a binary number, simply replace each octal digit with the appropriate three bits.

EXAMPLE 2-31

Convert each of the following octal numbers to binary:

(a) 13_8 (b) 25_8 (c) 140_8 (d) 7526_8

Solution

(a) $\begin{array}{cc} 1 & 3 \\ \downarrow & \downarrow \\ \underbrace{001} & \underbrace{011} \end{array}$ (b) $\begin{array}{cc} 2 & 5 \\ \downarrow & \downarrow \\ \underbrace{010} & \underbrace{101} \end{array}$ (c) $\begin{array}{ccc} 1 & 4 & 0 \\ \downarrow & \downarrow & \downarrow \\ \underbrace{001} & \underbrace{110} & \underbrace{000} \end{array}$ (d) $\begin{array}{cccc} 7 & 5 & 2 & 6 \\ \downarrow & \downarrow & \downarrow & \downarrow \\ \underbrace{111} & \underbrace{101} & \underbrace{010} & \underbrace{110} \end{array}$

Binary-to-Octal Conversion

Group bits into 3 from right (add leading zeros if needed) and convert each group to an octal digit.

EXAMPLE 2-32 (a) $110101_2 \rightarrow ?$

$$\begin{array}{ccc} 110 & 101 & \\ \downarrow & \downarrow & \\ 6 & 5 & \end{array} = 65_8$$

EXAMPLE 2-32 (b) $101111001_2 \rightarrow ?$

$$\begin{array}{ccc} 101 & 111 & 001 \\ \downarrow & \downarrow & \downarrow \\ 5 & 7 & 1 \end{array} = 571_8$$

EXAMPLE 2-32 (c) $100110011010_2 \rightarrow ?$

$$\begin{array}{cccc} 100 & 110 & 011 & 010 \\ \downarrow & \downarrow & \downarrow & \downarrow \\ 4 & 6 & 3 & 2 \end{array} = 4632_8$$

EXAMPLE 2-32 (d) $11010000100_2 \rightarrow ?$

$$\begin{array}{cccc} 0 & 011 & 010 & 000 & 100 \\ \downarrow & \downarrow & \downarrow & \downarrow & \downarrow \\ 3 & 2 & 0 & 4 & \end{array} = 3204_8$$



Related Problem

Convert 1010101000111110010_2 to octal.

10. Binary Coded Decimal (BCD)

- In BCD, 4 bits represent each decimal digit.
- Binary coded decimal (BCD) is a way to express each of the decimal digits with a binary code.
- There are only ten code groups in the BCD system, so it is very easy to convert between decimal and BCD.
- Because we like to read and write in decimal, the BCD code provides an excellent interface to binary systems.

The 8421 BCD Code

Each decimal digit (0–9) is represented by a 4-bit binary code with weights 8, 4, 2, 1 (2^3 , 2^2 , 2^1 , 2^0).

- The 8421 code is a type of BCD (binary coded decimal) code.
- Each decimal digit 0–9 is represented by 4 bits.
- Easy conversion between 8421 code and decimal is a key advantage.
- The 8421 code is the most common BCD code (used unless stated otherwise).

TABLE 2–5

Decimal/BCD conversion.

Decimal	0	1	2	3	4	5	6	7	8	9
BCD (8421)	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001



Key Point

8421 $\rightarrow$ bit weights: 8, 4, 2, 1.



Related Problem

Convert 274_{10} to 8421 BCD.

Invalid Codes (8421 BCD)

With 4 bits, 16 combinations exist, but in 8421 BCD only 10 (0–9) are valid.

Key Points

- 1 Invalid codes: 1010, 1011, 1100, 1101, 1110, 1111.
- 2 To write a decimal number in BCD, replace each digit with its 4-bit code.
- 3 Only digits 0–9 are used.

EXAMPLE 2–33

Convert to BCD.

(a) $35 \rightarrow 0011\ 0101$

$\begin{array}{c} 3 \qquad 5 \\ \downarrow \quad \downarrow \\ \underbrace{0011} \quad \underbrace{0101} \end{array}$

(b) $98 \rightarrow 1001\ 1000$

$\begin{array}{c} 9 \qquad 8 \\ \downarrow \quad \downarrow \\ \underbrace{1001} \quad \underbrace{1000} \end{array}$

(c) $170 \rightarrow 0001\ 0111\ 0000$

$\begin{array}{c} 1 \qquad 7 \qquad 0 \\ \downarrow \quad \downarrow \quad \downarrow \\ \underbrace{0001} \quad \underbrace{0111} \quad \underbrace{0000} \end{array}$

(d) $2469 \rightarrow 0010\ 0100\ 0110\ 1001$

$\begin{array}{c} 2 \qquad 4 \qquad 6 \qquad 9 \\ \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \\ \underbrace{0010} \quad \underbrace{0100} \quad \underbrace{0110} \quad \underbrace{1001} \end{array}$



Related Problem

Convert 9673 to BCD.

BCD to Decimal (Invalid Codes)

Group into 4 bits from the right and convert each group to its decimal digit (only 0–9 are valid).

EXAMPLE 2–34

Convert to decimal.

(a)

10000110
└───┬───┘
1000 0110
↓ ↓
8 6

= **86**

(b)

001101010001
└───┬───┬───┘
0011 0101 0001
↓ ↓ ↓
3 5 1

= **351**

(c)

1001010001110000
└───┬───┬───┬───┘
1001 0100 0111 0000
↓ ↓ ↓ ↓
9 4 7 0

= **9470**



Related Problem

Convert 10000010001001110110 to decimal.

BCD Addition

Add two BCD numbers: if a 4-bit sum > 9 or there is a carry, add 6 (0110) and carry to next digit.

Steps

- 1 Add using binary rules.
- 2 If the 4-bit sum ≤ 9 , it is valid.
- 3 If the 4-bit sum > 9 or carry occurs, add 6 (0110) and carry to the next digit.

EXAMPLE

Add 57 and 28 in BCD.

Step 1. Add (binary)

$$\begin{array}{r} \text{5} \quad \text{7} \\ 0101 \ 0111 \\ \text{2} \quad \text{8} \\ + 0010 \ 1000 \\ \hline 0111 \ 1111 \end{array}$$

Step 2. Check each digit

Right digit: $1111 (15) > 9$

$$+ 0110$$

$$\begin{array}{r} 1 \ 0101 \\ \text{carry 1} \end{array}$$

Left digit: $0111 + 1 = 1000 (8) \leq 9$

1000

Result: **85**



Related Problem

Add 46 and 79 in BCD.

EXAMPLE 2-35

Add the following BCD numbers.

(a) $0011 + 0100$

$$\begin{array}{r} 0011 \quad (3) \\ + 0100 \quad (4) \\ \hline 0111 \quad (7) \end{array}$$

(b) $00100011 + 00010101$

$$\begin{array}{r} 0010 \quad 0011 \quad (23) \\ + 0001 \quad 0101 \quad (15) \\ \hline 0011 \quad 1000 \quad (38) \end{array}$$

(c) $10000110 + 00010011$

$$\begin{array}{r} 1000 \quad 0110 \quad (86) \\ + 0001 \quad 0011 \quad (13) \\ \hline 1001 \quad 1001 \quad (99) \end{array}$$

(d) $010001010000 + 010000010111$

$$\begin{array}{r} 0100 \quad 0101 \quad 0000 \quad (450) \\ + 0100 \quad 0001 \quad 0111 \quad (417) \\ \hline 1000 \quad 0110 \quad 0111 \quad (867) \end{array}$$



Note

Each 4-bit sum ≤ 9 , so the results are valid BCD numbers.



Related Problem

Add $1001000001000011 + 0000100100100101$.

EXAMPLE 2-36

Add the following BCD numbers.

(a)

$1001 + 0100$

$$\begin{array}{r}
 1001 \quad (9) \\
 + 0100 \quad (4) \\
 \hline
 1101 \quad (> 9) \\
 + 0110 \\
 \hline
 \textcolor{red}{0001} \quad \textcolor{red}{0011} \\
 \downarrow \quad \downarrow \\
 1 \quad 3
 \end{array}$$

> 9, add 6.
Result = 13.

(b)

$1001 + 1001$

$$\begin{array}{r}
 1001 \quad (9) \\
 + 1001 \quad (9) \\
 \hline
 1 \ 0010 \quad (\text{carry, } > 9) \\
 + 0110 \\
 \hline
 \textcolor{red}{0001} \quad \textcolor{red}{1000} \\
 \downarrow \quad \downarrow \\
 1 \quad 8
 \end{array}$$

Carry, add 6.
Result = 18.

(c)

$00010110 + 00010101$

$$\begin{array}{r}
 0001 \ 0110 \quad (16) \\
 + 0001 \ 0101 \quad (15) \\
 \hline
 0010 \ 1011 \quad (> 9) \\
 + \quad \quad 0110 \\
 \hline
 \textcolor{red}{0011} \quad \textcolor{red}{0001} \\
 \downarrow \quad \downarrow \\
 3 \quad 1
 \end{array}$$

Right group > 9,
add 6 (carry 1).
Result = 31.

(d)

$01100111 + 01010011$

$$\begin{array}{r}
 0110 \ 0111 \quad (67) \\
 + 0101 \ 0011 \quad (53) \\
 \hline
 1011 \ 1010 \quad (> 9) \\
 + 0110 \ 0110 \\
 \hline
 \textcolor{red}{0001} \quad \textcolor{red}{0010} \quad \textcolor{red}{0000} \\
 \downarrow \quad \downarrow \quad \downarrow \\
 1 \quad 2 \quad 0
 \end{array}$$

Both groups > 9,
add 6 to each.
Result = 120.

**Related Problem**

Add the BCD numbers: 01001000 + 00110100.

The Gray Code

Gray code is unweighted and has only one bit change between successive codes.

Key Points

- 1** Unweighted; not an arithmetic code.
- 2** Only one bit changes between adjacent codes.
- 3** Useful in position encoders and other applications.

TABLE 2–6

Four-bit Gray code.

Decimal	Binary	Gray Code	Decimal	Binary	Gray Code
0	0000	0000	8	1000	1100
1	0001	0001	9	1001	1101
2	0010	0011	10	1010	1111
3	0011	0010	11	1011	1110
4	0100	0110	12	1100	1010
5	0101	0111	13	1101	1011
6	0110	0101	14	1110	1001
7	0111	0100	15	1111	1000



Main Idea

Gray code offers a single-bit change between successive values.

Binary-to-Gray Code Conversion

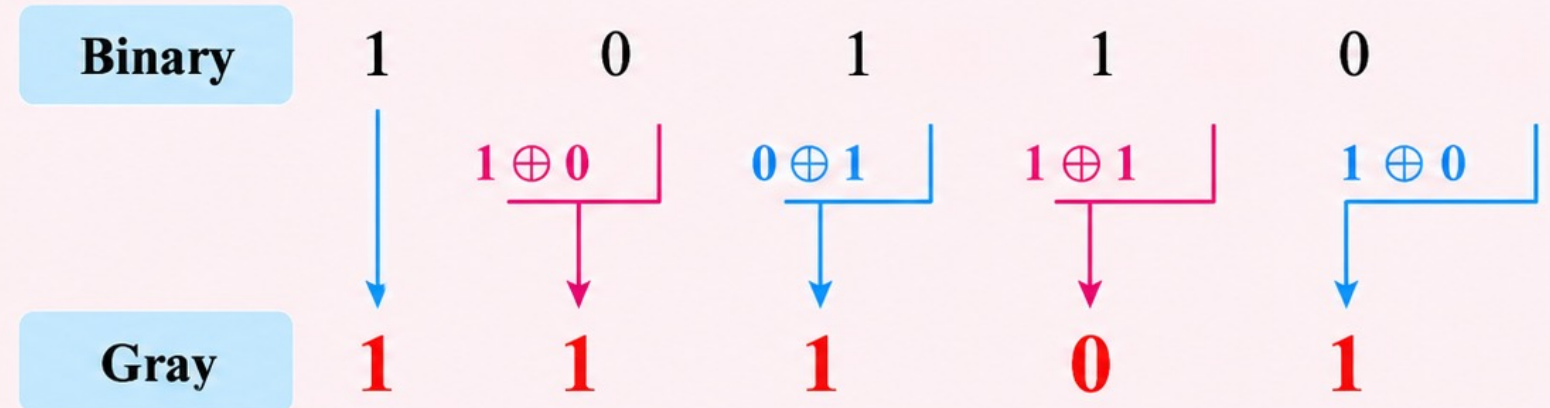
Keep the leftmost bit; each next Gray bit = XOR of adjacent binary bits (ignore carries).

Steps

- 1 Leftmost Gray bit = leftmost binary bit.
- 2 XOR each adjacent pair of binary bits.
- 3 Discard carries.

Example

Convert 10110 to Gray code.



Note

Gray code changes only one bit between successive values.

Gray-to-binary Code Conversion

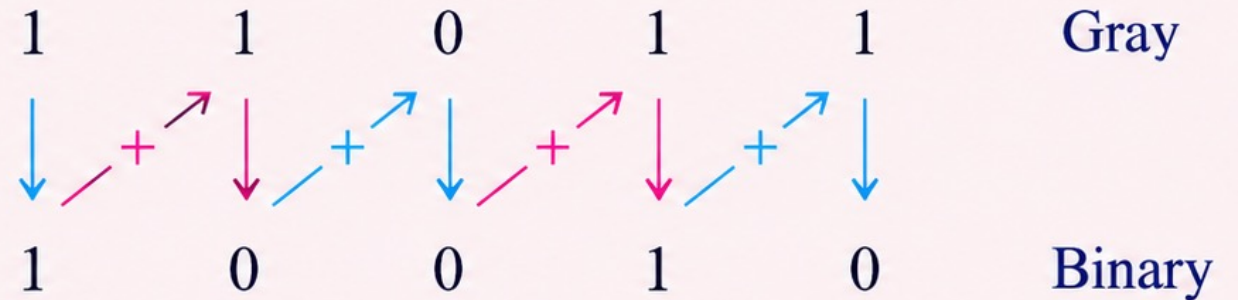
Keep the leftmost bit; each next binary bit = previous binary bit XOR next Gray bit.

Steps

- 1 Leftmost binary bit = leftmost Gray bit.
- 2 XOR with next Gray bit to get each binary bit.
- 3 Discard carries.

Example

Convert 11011 to binary.



Note

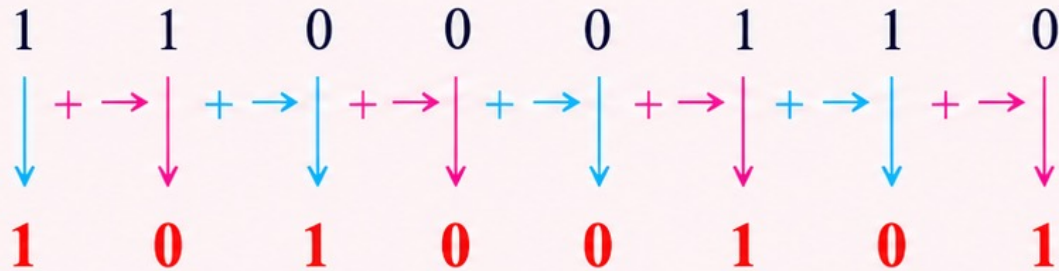
Each binary bit is the **XOR** of the previous binary bit and the next Gray bit.

EXAMPLE 2-37

- (a) Convert the binary number 11000110 to Gray code.
(b) Convert the Gray code 10101111 to binary.

(a) Binary to Gray code

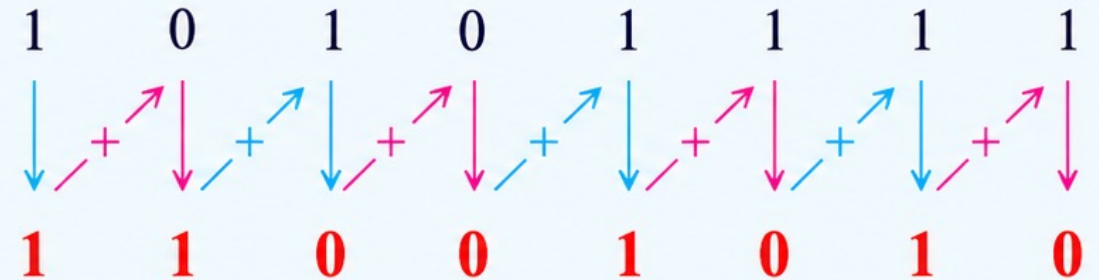
11000110 → ?



Gray code = **10100101**

(b) Gray code to binary

10101111 → ?



Binary number = **11001010**



Related Problem

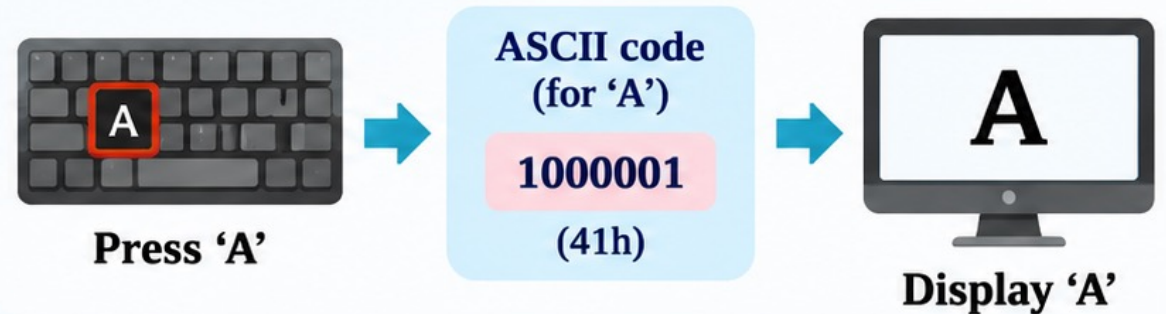
- (a) Convert binary 101101 to Gray code. (b) Convert Gray code 100111 to binary.

ASCII

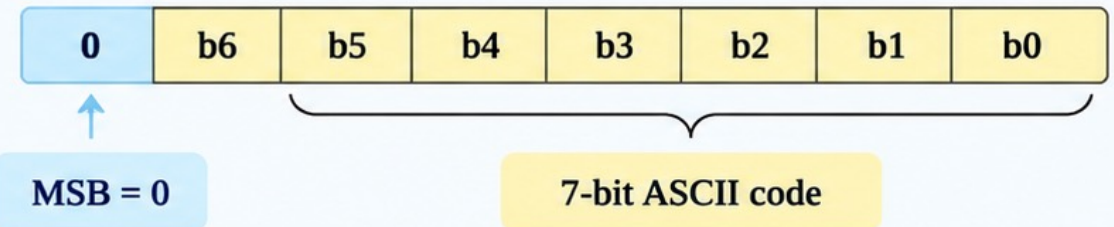
American **S**tandard **C**ode for **I**nformation **I**nterchange

- 1 ASCII is a standard alphanumeric code used in most computers and electronic devices.
- 2 Keyboards use ASCII to encode letters, numbers and control commands.
- 3 128 characters represented by 7 bits (often written as 8 bits with MSB = 0). Range: 00 – 7F (hex).
- 4 First 32 characters are non-graphic control codes (not displayed).
- 5 Others are printable characters: uppercase & lowercase letters, digits, punctuation and common symbols.

How it works?



ASCII Code Structure



Total: 128 characters (00 – 7F hex)



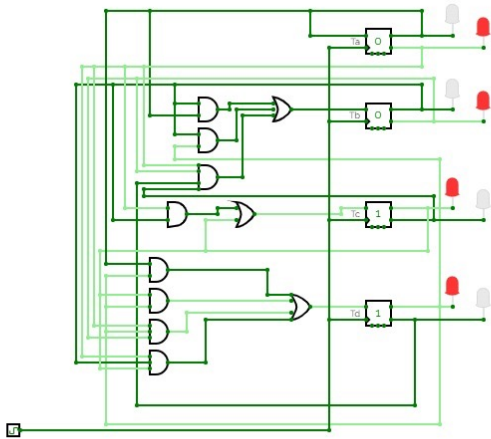
Vietnam National University HCMC
International University



School of
Electrical Engineering

THE END

Lecture 2: Number Systems, Operations, and Codes



INSTRUCTOR: Dr. Vuong Quoc Bao